Predict, Don't React: Value-Based Safety Forecasting for LLM Streaming

Pride Kavumba[1] **Koki Wataoka**[1] **Huy H. Nguyen**[1] **Jiaxuan Li**[1] **Masaya Ohagi**[1*]
[1]SB Intuitions Corp.
{pride.kavumba,koki.wataoka,hong.huy.nguyen,jiaxuan.li}@sbintuitions.co.jp

arXiv:2604.03962v1 [cs.CL] 5 Apr 2026

Abstract

In many practical LLM deployments, a single guardrail is used for both input and output moderation. Input moderation operates on fully observed text, whereas streaming output moderation requires safety decisions to be made over partial generations. Existing text-based streaming guardrails commonly frame this output-side problem as boundary detection, training models to identify the earliest prefix at which a response has already become unsafe. In this work, we introduce StreamGuard, a unified model-agnostic streaming guardrail that instead formulates moderation as a forecasting problem: given a partial prefix, the model predicts the expected harmfulness of likely future continuations. We supervise this prediction using Monte Carlo rollouts, which enables early intervention without requiring exact token-level boundary annotations.

Across standard safety benchmarks, StreamGuard performs strongly both for input moderation and for streaming output moderation. At the 8B scale, StreamGuard improves aggregated input-moderation F1 from 86.7 to 88.2 and aggregated streaming output-moderation F1 from 80.4 to 81.9 relative to Qwen3Guard-Stream-8B-strict. On the QWENGUARDTEST `response_loc` streaming benchmark, StreamGuard reaches 97.5 F1, 95.1 recall, and 92.6% on-time intervention, compared to 95.9 F1, 92.1 recall, and 89.9% for Qwen3Guard-Stream-8B-strict, while reducing the miss rate from 7.9% to 4.9%. We further show that forecasting-based supervision transfers effectively across tokenizers and model families: with transferred targets, Gemma3-StreamGuard-1B reaches 81.3 output-moderation F1, 98.2 streaming F1, and a 3.5% miss rate. These results show that forecasting future risk enables effective low-latency streaming moderation without exact boundary labels.[1]

1 Introduction

Safety guardrails are now a standard component of Large Language Model (LLM) deployments, mitigating risks such as harmful instructions and toxic content (Carlini et al., 2023; Wei et al., 2023; Zhao et al., 2024). In practice, a single guardrail is often deployed to handle both input and output moderation, but the two settings differ in observability. Input moderation operates on fully observed text, whereas output moderation in streaming systems requires decisions over partial prefixes before the final completion is available, creating a tension between safety and responsiveness.

Text moderators such as LlamaGuard (Inan et al., 2023) are designed for full-sequence classification and are therefore most naturally applied after generation has completed. In a streaming product, that leaves an undesirable choice: either hold back the response until moderation finishes, increasing latency, or display tokens before the safety decision is finalized, creating an exposure window during which unsafe content may already have been shown. White-box defenses address this problem by attaching classifiers to the generator's

[*]Currently at Sakana AI
[1]Upon acceptance, we plan to release the research artifacts associated with this work.

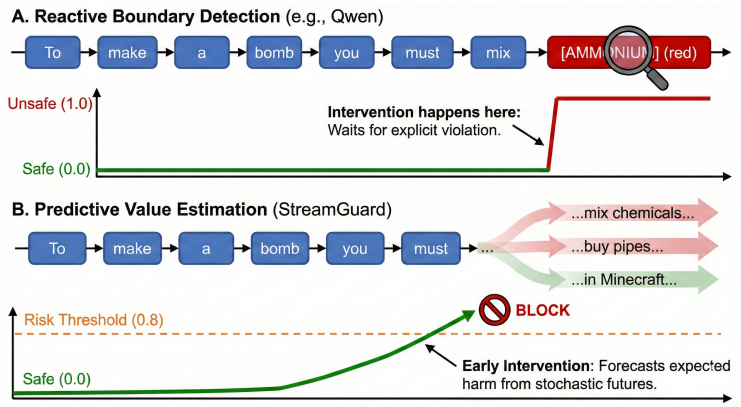


Figure 1: Boundary detection versus forecasting in streaming moderation. Top: Boundary-detection methods evaluate whether the *current prefix* has already crossed an unsafe boundary, so intervention is tied to when unsafe content becomes explicit in the text. Bottom: StreamGuard instead scores each prefix by its *expected future harmfulness*, predicting whether likely continuations are likely to lead to an unsafe completion. This supports early intervention in streaming settings without requiring exact token-level boundary annotations.

internal activations (Li et al., 2025), but these approaches require access to hidden states and are coupled to particular model architectures and checkpoints, which limits portability.

Recent text-based streaming guardrails address this gap by framing moderation as a *boundary detection* problem: the model is trained to identify the earliest prefix at which a response has already become unsafe. This is a natural formulation, and systems such as Qwen3Guard-Stream (Zhao et al., 2025) provide a strong reference point for streaming moderation. At the same time, exact boundary supervision is expensive to construct, depends on the rollout-and-judge pipeline used to define the boundary, and is tied to tokenizer-specific coordinates. These practical costs motivate a different approach to streaming moderation.

In this work, we introduce StreamGuard, a unified model-agnostic guardrail for streaming LLM systems that instead formulates moderation as a *forecasting* problem. As illustrated in Figure 1, StreamGuard estimates the expected harmfulness of likely future continuations from the current prefix. We train this predictor using Monte Carlo rollouts: for each prefix, we sample possible futures, score them with a safety judge, and use the resulting future-risk estimate as supervision. A key advantage of this formulation is portability. Because supervision is defined over text prefixes with soft future-risk targets rather than tokenizer-specific boundary indices, the resulting training data transfers naturally across tokenizers and model families.

Across our evaluations, StreamGuard delivers strong performance both as an input moderator and as a streaming response guardrail. At the 8B scale, StreamGuard improves aggregated input-moderation F1 from 86.7 to 88.2 and aggregated streaming output-moderation F1 from 80.4 to 81.9 relative to Qwen3Guard-Stream-8B-strict. On the QWENGUARDTEST `response_loc` streaming benchmark, the 8B StreamGuard model reaches 97.5 F1, 95.1 recall, and 92.6% on-time intervention, compared to 95.9 F1, 92.1 recall, and 89.9% for Qwen3Guard-Stream-8B-strict, while reducing the miss rate from 7.9% to 4.9%. These results show that strong end-to-end streaming moderation can be obtained without exact boundary labels, while retaining portability across model families.

Our main contributions are as follows:

- We introduce StreamGuard, a unified guardrail for prompt and streaming output moderation, with a forecasting-based formulation for streaming output moderation that provides a model-agnostic alternative to boundary supervision.

- We show that forecasting yields strong end-to-end moderation performance, improving both standard input/output moderation scores and streaming unsafe-event detection against a strong published boundary-based reference.
- We demonstrate that forecasting supervision transfers across model families and tokenizers, including to Gemma (Team et al., 2025) and Qwen (Yang et al., 2025) backbones, and we provide ablations characterizing how rollout construction, aggregation, filtering, and supervision density affect performance and annotation cost.

2 Methodology

StreamGuard's core methodological idea is to cast streaming output moderation as a *forecasting* problem. Given a partially generated response, the model estimates a scalar *future-risk* score: the expected harmfulness of plausible continuations conditioned on the prompt and the observed prefix.

2.1 Prefix-Conditioned Safety Value

Let x denote a user prompt and let $y = \{y_1, y_2, \ldots, y_T\}$ denote a model response. For each observed prefix $y_{\le t}$, we define the target value as the expected harmfulness of plausible future completions:

$$V^*(x, y_{\le t}) = \mathbb{E}_{\tilde{y}_{t+1:T} \sim q(\cdot \mid x, y_{\le t})} \left[\mathcal{O}(x, y_{\le t} \,\|\, \tilde{y}_{t+1:T}) \right], \tag{1}$$

where $q(\cdot \mid x, y_{\le t})$ is a continuation distribution over possible futures and $\mathcal{O}(\cdot)$ is a safety judge that maps a completed response to a score in $[0, 1]$.

Unlike boundary-detection approaches such as Qwen3Guard-Stream (Zhao et al., 2025), which predict the first prefix that is already explicitly unsafe, we estimate the expected risk of continuing from the current prefix. For example, for *"To build a bomb, first you must"*, a boundary detector may still predict safe because no explicit violation has yet appeared, whereas our objective assigns a high score if likely continuations are harmful. This framing also avoids propagating the final response label to all earlier prefixes, which would incorrectly mark generic scaffolds such as *"I"*, *"The"*, or *"This"* as unsafe whenever they occur in an unsafe completion.

2.2 Monte Carlo Future-Risk Targets

The expectation in Equation (1) is intractable to compute exactly, so we approximate it with Monte Carlo rollouts. For a supervised prefix $y_{\le t}$, we sample M continuations

$$\mathcal{R}_t = \left\{ \tilde{y}^{(m)}_{t+1:T} \right\}_{m=1}^{M}, \qquad \tilde{y}^{(m)}_{t+1:T} \sim q(\cdot \mid x, y_{\le t}). \tag{2}$$

We then score each completed continuation with the offline guardrail and average the results:

$$\hat{v}_t = \frac{1}{M} \sum_{m=1}^{M} \mathcal{O}\left(x, y_{\le t} \,\|\, \tilde{y}^{(m)}_{t+1:T} \right). \tag{3}$$

The resulting target $\hat{v}_t \in [0, 1]$ is a soft estimate of conditional future risk. Prefixes whose continuations are usually benign receive low targets, even if the realized completion is unsafe, while prefixes that reliably lead to harmful continuations receive high targets before explicit policy-violating text appears.

Rollout proposal distribution. The continuation distribution q defines what counts as a plausible future. In the simplest case, q is induced by a single generator model. More generally, we allow q to be a mixture of generators:

$$q(\tilde{y} \mid x, y_{\le t}) = \sum_{j=1}^{J} \alpha_j \, \pi_j(\tilde{y} \mid x, y_{\le t}), \qquad \sum_{j=1}^{J} \alpha_j = 1, \tag{4}$$

where each π_j is a rollout generator and α_j is its mixture weight.

Using a mixture reduces dependence on the continuation bias of any single model and broadens the futures reflected in the target. Because the supervision depends on q, rollout generators should neither assign uniformly high risk to generic early prefixes nor suppress risk through excessive refusal behavior. We treat rollout choice and mixture as target-construction decisions and analyze them in Section 5.

2.3 Training Objective

StreamGuard predicts a scalar risk score for each observed prefix:

$$V_\theta(x, y_{\le t}) = \sigma(W h_t + b), \tag{5}$$

where h_t is the representation of the current prefix and σ is the sigmoid function.

Let $\mathcal{T}(x, y)$ denote the set of token positions selected for supervision for example (x, y), and let $\ell_{\text{gt}} \in \{0, 1\}$ denote the example-level label for the completed response. For intermediate positions $t < T$, the target is the Monte Carlo estimate $\hat{v}_t$; for the terminal position, the target is the hard label ℓ_{gt}. Writing the target uniformly as $\tilde{v}_t$, we minimize binary cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{(x,y)\in\mathcal{B}} \frac{1}{|\mathcal{T}(x,y)|} \sum_{t\in\mathcal{T}(x,y)} \mathcal{L}_{\text{BCE}}\left(V_\theta(x, y_{\le t}), \tilde{v}_t\right). \tag{6}$$

This objective trains the model to estimate future risk throughout the generation trajectory. At inference time, StreamGuard outputs a risk score for each streamed prefix. For the main results, we use a fixed reference target-construction setup and analyze rollout-source composition, score aggregation, and supervision density in Section 5.

3 Experimental Setup

3.1 Datasets and Evaluation

We evaluate StreamGuard on input moderation, output moderation, overblocking on challenging benign responses, streaming intervention timing, and transfer across tokenizers and model families. Input moderation is evaluated on fully observed text, while output moderation is evaluated in a simulated streaming setting in which the model processes outputs one token at a time. For input and output moderation, we follow the test-split settings used by Han et al. (2024).

Input moderation. We evaluate unsafe-prompt detection on ToxicChat (Lin et al., 2023), OpenAIModeration (Markov et al., 2023), Aegis (Ghosh et al., 2024), Aegis2.0 (Ghosh et al., 2025), SimpleSafetyTests (Vidgen et al., 2024), HarmBench (Mazeika et al., 2024), and WildGuardTest (Han et al., 2024). Following prior guardrail work (Inan et al., 2023; Han et al., 2024; Zhao et al., 2025), we report F1.

Output moderation. We evaluate unsafe-response detection on HarmBench, SafeRLHF (Dai et al., 2023), BeaverTails (Ji et al., 2023), XSTest-Resp (Han et al., 2024), WildGuardTest, and Aegis2.0. We report F1. All experiments other than input moderation use the debounced rule from Qwen3Guard-Stream (Zhao et al., 2025): a response is counted as unsafe after two consecutive unsafe predictions.

Overblocking. We evaluate overblocking on XSTest-Resp. Because it contains benign responses with strong lexical overlap to unsafe requests, it is a stress test for whether a moderator incorrectly flags safe responses based on surface cues. We report false positive rate (FPR) on the benign subset.